Cryptography and Technical Fundamentals of Blockchain (BTC/ETH)

Leon Godtfriedsen[*] Sahand Sarbaz[†] Josh Davidson[‡]
Jose Garrido Boix[§]

September 2025

[*]Lead author, University of Edinburgh
[†]Co-author, University of Edinburgh
[‡]Co-author, University of Edinburgh
[§]Co-author, University of Edinburgh

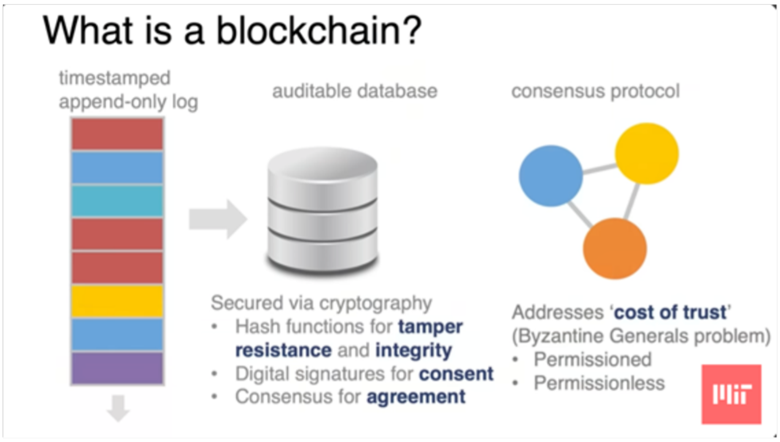
Week 2 - Cryptographic Functions and Blockchain Technology (BTC/ETH)

As mentioned in the previous section, BTC as a cryptocurrency has 3 core elements. These being; cryptography, consensus mechanisms and a ledger.
Below we will initially discuss the cryptography of Hash functions, which compresses and masks all the transaction data. We will also discuss the asymmetric cryptography element, which allows for private and public key generation, facilitating confidential transactions.
These elements provide the cryptographic security needed for digital cash. This Is combined with the consensus algorithm, which for BTC is Proof of Work, solving for the Byzantine Fault Argument aka. bad actors, allowing for decentralisation. This is combined with the ledger, providing the elements needed for digital currencies.
Some useful background information and reminders: Key features of BTC:

- Hash Function (Cryptography)
- Private and Public Key (asymmetric cryptography): Digital Signatures
- Time stamped append only logs
- Consensus Algorithm
- Light and Full Nodes
- Peer to Peer
- Block heads, Merkle Tree Structure (way to compress data)
- Nonce, (random number once)

Ledgers can be two types: Transaction (BTC) / Balance (ETH):

- BTC has a max supply reached in 2040 (21 million), every ten minutes a block is rewarded, worth x amount of btc, which is halved every 4 years.
- Miner reward halves every 4 years, initially 100, 50, 25, 12.5 etc. The cost of electricity has to be low to make it worth it.
- To hedge bets and guarantee some income mining pools are created.
- This is an issue as it tends towards another form of centralisation



- Miner reward halves every 4 years, initially 100, 50, 25, 12.5 etc. etc. Cost of electricity has to be low to make it worth it.
- To hedge bets and guarantee some income mining pools are created.
- This is an issue as it tends towards another form of centralisation

Cryptographic Elements

Hash Functions

Hash Function: Digital Fingerprint for data What a hash function will do is:

- Take input x of any size into a fixed sized output y – y is the hash
- Deterministic – Always same hash for same x (cause of immutability)
- SHA256, BTC's hash function, compresses thousands of BTC tx's into a 256 bit line, placed on the block
- This makes the data compressed and extremely efficient. Its deterministic traits also help remove bad actors, as any slight change in transaction data will have huge effects.

Crypto properties:

- Preimage resistant: Only one way, cannot determine Y from X.
  - Infeasible - not impossible - due to brute force. (odds are $1/10\hat{4}0$): Collision Resistant.

- Avalanche effect: change a hash slightly, whole chain changes. Easy to detect manipulation.

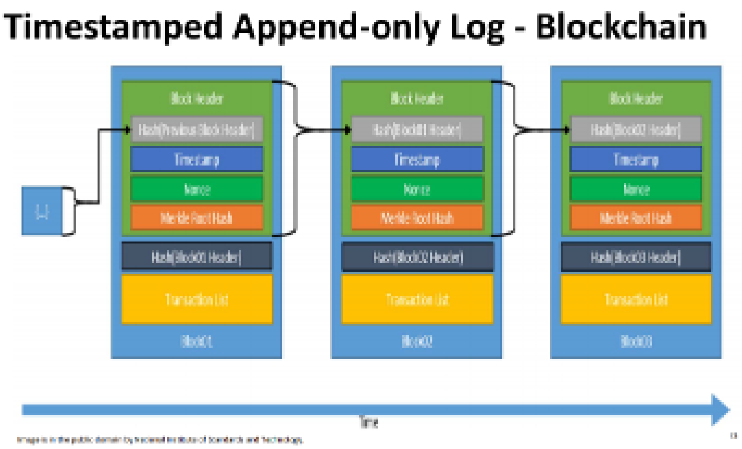
In BTC Hash functions are used as pointers, pointing to the next block creating the chain.
The BTC Block has two key parts: The Block Header (ID), and the Transaction List.
Block Header Components

- **Hash of Previous Block Header**
  - Every block contains the hash of the block before it.
  - This is how blocks are "chained" → if someone tampers with an earlier block, its hash changes, breaking the chain.
- **Timestamp**
  - Records when the block was created.
  - Included in the header, so it influences the block's hash.
- **Nonce**
  - A number miners keep changing until the hash of the header meets the difficulty requirement (enough leading zeros).
  - This is the Proof-of-Work mechanism.
- **Merkle Root Hash**
  - A single hash summarizing all the transactions in the block, generated from the **Merkle tree** of transactions.
  - Ensures that if even one transaction changes, the Merkle root changes, and therefore the entire block's hash changes.

Transaction List

- The full list of transactions inside the block.
- These are not hashed directly into the header (too large) but instead condensed into the Merkle root for efficiency.//

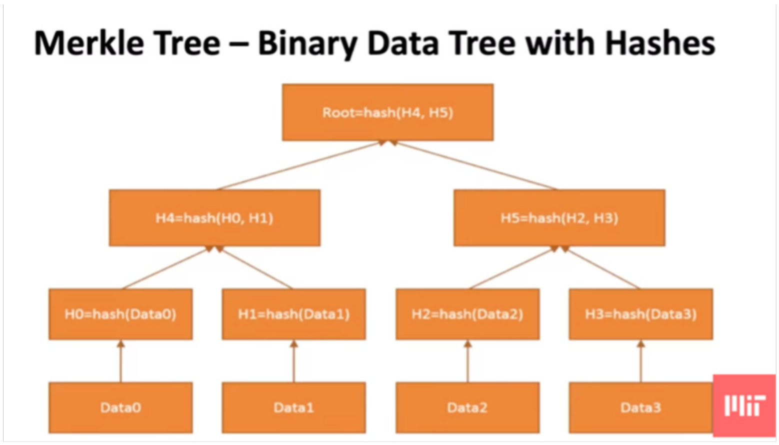
In essence the cycle of the block is:



- All transactions are placed in a Merkle root hash.
- These transactions are placed in the Block Header, alongside the other elements of a blockchain (timestamp, nonce, Merkle root hash, and previous hash).
- The entire block header is then hashed and placed in the next block, creating an immutable chain (owing to the avalanche effect).

Light nodes only contain the block headers, whilst the full nodes contain the total transaction list (UTXO).

- The Nonce acts as the difficulty target for the miners. More miners → Harder.

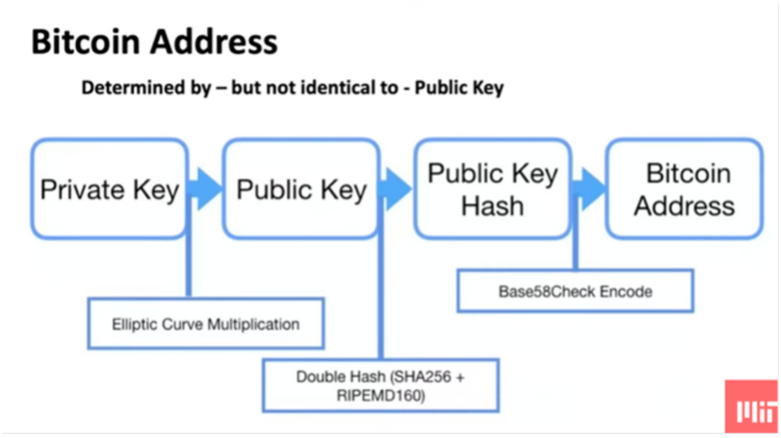


Merkle Tree: Binary Data Tree which is a composition of many hashes, which are tx's (2000)

As seen above, the cryptographic element of Hash Functions allows for data compression and the linking of the blocks, allowing for the creation of an immutable ledger.

Asymmetric Cryptography and Digital Signatures

Asymmetric cryptography & digital signatures are what make Bitcoin work. The private key gives you control over your BTC, the public key/address lets others send you BTC, and the digital signature lets the network verify transactions without needing to trust you personally.



Digital Signature Algorithms generate a key pair:

- A private key is generated from random digits, where the public key is then derived from the private key.
- For BTC this is called the Elliptic Curve Digital Signature Algorithm.
  - The **private key** is secret; whoever controls it controls the funds.
  - The **public key** can be safely shared; it proves ownership and allows others to verify signatures.
  - It's computationally infeasible to derive the private key from the public key.

Digital Signatures

- **Signing:**
  - The owner uses their private key to sign a transaction (basically proving: "I authorize this transfer").

  - The digital signature depends on both the message (transaction data) and the private key, so it's unique to that specific transaction.

- **Verification:**
  - Anyone on the Bitcoin network can check the signature with the **public key**.
  - If valid, it proves the transaction really came from the holder of the private key — without revealing the **private key** itself.

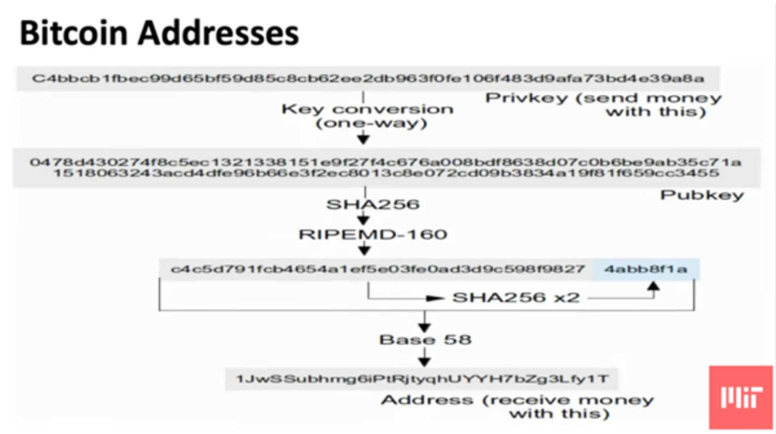
Step by Step explanation for BTC:

1. **Address Generation**
   (a) Your Bitcoin "address" is derived from your public key (after hashing), which is derived from your private key.
   (b) Anyone can send BTC to this address, but only the private key holder can move it.

2. **Making a Transaction**
   (a) When you spend BTC, you create a transaction message
   (b) You sign it with your private key → this creates a digital signature.

3. **Verification by the Network**
   (a) All Bitcoin nodes take your signed transaction.
   (b) They check the signature against the public key linked to your BTC.
   (c) If the math checks out, the network knows:
      i. You must own the private key (so you're the rightful spender).
      ii. The transaction data wasn't altered.

4. **Security**
   (a) No one can forge your signature without your private key.
   (b) No one can reverse-engineer your private key from your public key.
   (c) This ensures trustless, decentralized security — no banks or middlemen required.



Example Transaction for BTC:
Full Process from A (sender) to B(receiver):

1. **A:** A creates a transaction: "Send 1 BTC to B's address".
2. A signs the transaction with A's private key.
3. A broadcasts the signed transaction to the Bitcoin network (miners and full nodes).
4. **The Bitcoin Network:** Nodes verify the signature using A's public key This proves A authorized the transaction.
5. Nodes check A really does have the BTC to spend (i.e., unspent output).
6. If valid, miners include the transaction in a block.
7. That block is added to the blockchain.
8. **B:** B does not have to verify anything manually because:
   (a) B's Bitcoin wallet 'listens' to the blockchain.
   (b) When it sees BTC sent to B's public address, it updates B's balance.
9. Now B can spend the BTC later by signing a new transaction with B's private key.

Consensus Mechanism

Consensus Mechanism, in general, is meant to provide a solution to the Byzantine General Problem (BGP). The BGP describes the challenge of achieving consensus in a distributed system where participants (generals) must agree on a single course of action (attack or retreat) despite some participants (traitors) acting maliciously or unreliably, potentially sending false or contradictory information

- In relation to BTC, this means that all the nodes must agree on the same transactions that take place on the blockchain. This is the solution to the double spend problem.

Essentially, Satoshi Nakamoto invented a solution to a hard problem, that's easy to verify based on computer processing.
The reason the need for a consensus algorithm exists, is due to the lack of central authority. The consensus algorithm is what ensures:

- Everyone agrees on which transactions are valid.
- Only one chain of history is accepted (no double spending).
- Malicious or faulty nodes cannot break the system.

In relation to BTC, which uses Proof of Work (Nakamoto Consensus), the step by step process is:

- Transaction broadcast: Users send BTC $\rightarrow$ transactions are spread across the network.
- Block creation (mining):
  - Miners gather unconfirmed transactions.
  - They compete to solve a cryptographic puzzle:
    - H(block header) ¡ target, where H is SHA-256 hash.
  - This requires enormous random hashing — work.
- Block validation:
  - First miner to solve broadcasts their block.
  - Other nodes verify transactions + hash condition.
- Chain selection rule (Longest Chain Rule):

  - If two blocks appear, nodes follow the chain with the most cumulative work.
  - This ensures eventual agreement, even if forks happen.

- Finality (probabilistic):
  - A transaction is considered secure after 6 confirmations (≈ 1 hour).
  - Because rewriting 6 blocks would require astronomical computing power.

Why Proof of Work Works:

- Sybil resistance → anyone can spin up fake identities, but they can't fake computing power.
- Security → rewriting history requires ¿50% of total network hash power (the "51% attack").
- Decentralization → anyone can join as a miner if they contribute hardware.

Decentralised Networks: No central authority – How do we get a group of Generals to Attack at the same time? How do we prevent one general from committing treason?
Proof of Work:
If you change a former block, all future hashes will be invalidated. Solution to the Byzantine Fault Problem:

- Solves complicated mathematical puzzles to verify blocks, where miner that mines block is rewarded.
- This is then added to the chain and easily verified by the rest of the nodes.

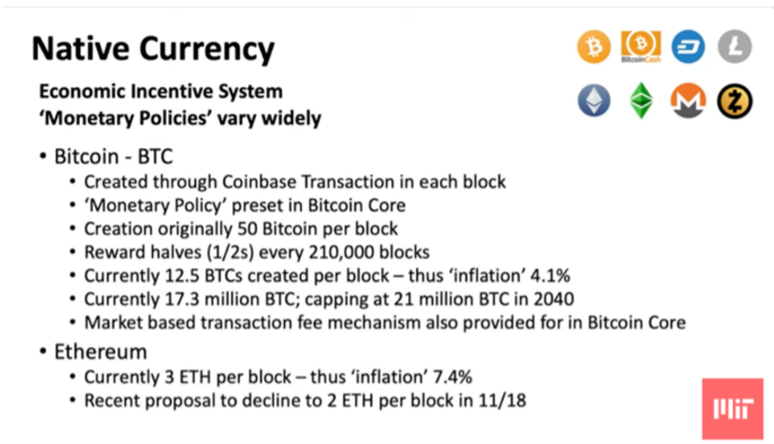
Can have orphan chains, which typically die off, but can also be hard forked.

- Bitcoin difficulty is based off leading zeroes in the hash
- Block should be mined every 10 minutes ... more than 10 trillion times harder to hash then 2009
- Difficulty changes every 2 weeks

Opportunity cost of electricity cost is very real – but there is cost associated with all central banks as well – manufacturing of physical currencies, office space, staff costs, etc. Computational cost is what guarantees safety and justifies value.

Bitcoin Mining Evolution:
CPU – GPU – ASIC: Gives far greater hash rate. Now in Tera Hashes $(10\hat{1}2)$
There have been a large variety of consensus mechanisms invented such as Proof of Work, Proof of Stake and Proof of History.
Network:

- Full nodes: Store blockchain & able to validate all transactions.
- Pruning Nodes – prune transactions after validating.
- Light weight node: Stores just block headers (relies on full nodes).
- Miners: don't need to operate a full node (mining pools are Achilles heel)
  - 10000 nodes
- Mining Pool operators
- Wallet
- Memory Pool (Mempool)



Risks of mining pools: 51% attack

- Only need one full node in a mining pool (increases centralisation somewhat).

Native Currency is the incentive system for the consensus mechanism, i.e. network security is aligned with economic interest.
Rough overview of different consensus algorithms:
Proof of Work (PoW)

- Miners solve cryptographic puzzles to add blocks.
- Example: Bitcoin.
- Pro: Secure, decentralized
- Con: Energy-intensive, slow

Proof of Stake (PoS)

- Validators lock up coins ("stake") to be chosen to add blocks.
- Example: Ethereum, Cardano.
- Pro: Energy-efficient, faster finality
- Con: Risk of wealth concentration, complex rules

Delegated Proof of Stake (DPoS)

- Token holders vote for a small group of delegates who validate.
- Example: EOS, TRON.
- Pro: Very fast, scalable
- Con: Centralization risk (few validators in control)

Proof of Authority (PoA)

- Trusted entities/identities act as validators.
- Example: VeChain, enterprise/private chains.
- Pro: Efficient, predictable
- Con: Not decentralized (relies on trust)

Unspent Transaction Output (UTXO) and Transactions

The UTXO acts as a current account balance snapshot, providing the nodes with the overview of the individual account balances of different wallets.

- The blockchain itself is the ledger → it records all transactions from the beginning (genesis block).
- The **UTXO set** is more like a current snapshot of spendable coins:
  - It's a database (maintained by every full node) of all unspent outputs that exist at this moment.
  - Whenever a new block is added, nodes update the UTXO set:
    - Remove the UTXOs that were spent.
    - Add the new outputs created.

In essence, the blockchain is a giant book of all transactions (the ledger), the UTXO is the current account balances

- Blockchain = History — UTXO = Current Balance Sheet — UTXO is current wallet Balance, Block chain is bank statement history.

After a transaction is done, it is broadcasted on chain and joined into Merkle roots, a miner mines the block and then adds the transaction to the blockchain which is verified on chain with UTXO by full nodes. The more blocks added the more secure it is.